Fundamentos de Comunicação HTTP com Node.js e Express

Pedro O. Rabelo[1], **Rafael S. Rosa**[1], **Rian C. S. dos Santos**[1], **Christiam A. L. Assis**[1]

[1]Departamento de Ciências Exatas e da Terra – Universidade do Estado da Bahia (UNEB) – Salvador – BA – Brazil

Abstract. *This work aims to explore, in a practical way, the fundamental concepts of computer networks through the development of an application based on the client-server model. For this purpose, an HTTP server was implemented using the Node.js platform, with support for handling requests and communicating with clients through the web. The proposal seeks to demonstrate how information exchange occurs between client and server, highlighting the role of the HTTP protocol in the communication process.*
In addition, the project makes it possible to observe important aspects related to request behavior, response structure, and the interaction between different components of a web application. It also considers elements that influence how the server adapts its responses, such as information sent by the client during communication.
Thus, this work contributes to a better understanding of the basic principles that support the functioning of the internet, integrating theory and practice in an applied context. The results reinforce the importance of these concepts for the development of modern applications and for deeper studies in the field of computer networks.

Resumo. *Este trabalho tem como objetivo melhorar, de forma prática, os conceitos fundamentais de redes de computadores por meio do desenvolvimento de uma aplicação baseada no modelo cliente-servidor. Para isso, foi implementado um servidor HTTP utilizando a plataforma Node.js, com suporte ao tratamento de requisições e comunicação com clientes por meio da web. A proposta busca demonstrar, na prática, como ocorre a troca de informações entre cliente e servidor, evidenciando o papel do protocolo HTTP no processo de comunicação.*
Além disso, o projeto permite observar aspectos importantes relacionados ao comportamento das requisições, à estrutura das respostas e à interação entre diferentes componentes de uma aplicação web. Também são considerados elementos que influenciam a adaptação das respostas do servidor, como informações enviadas pelo cliente durante a comunicação.
Dessa forma, o trabalho contribui para a compreensão dos princípios básicos que sustentam o funcionamento da internet, integrando teoria e prática em um contexto aplicado. Os resultados obtidos reforçam a importância do entendimento desses conceitos para o desenvolvimento de aplicações modernas e para o estudo mais aprofundado da área de redes de computadores.

1. Introdução

O funcionamento das aplicações web modernas está diretamente relacionada à forma como ocorre a comunicação entre clientes e servidores. Essa comunicação é viabilizada principalmente pelo protocolo HTTP, que define como as requisições são enviadas

e como as respostas devem ser estruturadas [Fielding, R. and Reschke, J. 2014]. Compreender esse processo vai além do uso cotidiano da internet, sendo fundamental para o desenvolvimento e análise de sistemas distribuídos.

No modelo cliente-servidor, que é o modelo principal do nosso projeto, o cliente é responsável por iniciar a comunicação por meio de requisições, enquanto o servidor processa essas solicitações e retorna uma resposta adequada [Tanenbaum and Wetherall 2011]. Esse fluxo envolve diferentes tipos de requisições, como GET e POST, além da manipulação de dados e definição de rotas, que determinam como o servidor deve reagir a cada acesso. Esses elementos formam a base do funcionamento de APIs e aplicações web em geral.

Além da estrutura básica das requisições, o protocolo HTTP/1.1 introduz características importantes, como o uso de conexões persistentes [Fielding, R. and Reschke, J. 2014]. Esse recurso permite que múltiplas requisições sejam realizadas utilizando a mesma conexão, reduzindo o custo de estabelecimento de novas conexões e melhorando o desempenho da comunicação. Esse comportamento pode ser observado na prática ao analisar o tráfego entre cliente e servidor [Fielding, R. and Reschke, J. 2014].

Outro ponto relevante é o uso de cabeçalhos HTTP, que transportam informações adicionais durante a comunicação. Entre eles, destaca-se o User-Agent, que identifica características do cliente, como navegador e sistema operacional. Essas informações podem ser utilizadas pelo servidor para adaptar suas respostas ou registrar dados sobre os acessos realizados.

Com o objetivo de analisar esses conceitos de forma prática, este trabalho apresenta a implementação de um servidor HTTP utilizando Node.js, com o auxílio do framework Express. A aplicação desenvolvida permite definir rotas, tratar diferentes tipos de requisições e observar o comportamento da comunicação entre cliente e servidor em tempo real.

A partir dessa implementação, torna-se possível relacionar diretamente os conceitos teóricos estudados na disciplina com sua aplicação prática, evidenciando como as requisições são processadas e como as respostas são construídas. Dessa forma, o trabalho contribui para uma compreensão mais concreta do funcionamento das redes de computadores no contexto de aplicações web.

Por fim, o trabalho está organizado em seções que abordam o funcionamento do servidor, a definição de rotas e tipos de requisição, além de uma análise por meio de engenharia reversa, finalizando com as considerações sobre os resultados obtidos.

2. Tecnologias Utilizadas

Node.js

Ambiente de execução JavaScript no lado do servidor, responsável por executar o arquivo `server.js`. Utiliza um modelo assíncrono e orientado a eventos, permitindo o tratamento eficiente de múltiplas requisições.

Express.js

Framework para Node.js que facilita a criação de servidores HTTP e o gerenciamento de rotas. Embora não tenha sido utilizado diretamente na implementação, é amplamente empregado para simplificar o desenvolvimento de aplicações web.

HTTP/1.1

Protocolo de comunicação utilizado entre cliente e servidor, definindo o formato das requisições e respostas, além de suportar conexões persistentes.

Terminal / Linha de Comando

Utilizado para executar o servidor por meio do comando `node server.js`, além de exibir logs das requisições recebidas.

Navegador Web (Cliente)

Responsável por enviar requisições HTTP ao servidor e exibir as respostas, permitindo também a inspeção por meio de ferramentas de desenvolvedor.

3. Funcionamento do Servidor

O funcionamento do servidor segue o modelo cliente-servidor, utilizando HTTP/1.1 para realizar a comunicação entre o navegador e a aplicação.

Inicialmente, o servidor é iniciado através do arquivo server.js, executado com o comando node server.js. Nesse momento, o Node.js configura o ambiente de execução e, com o auxílio do Express.js, passa a escutar requisições na porta 3000.

Quando um cliente (navegador) acessa o endereço http://localhost:3000, uma requisição HTTP é enviada ao servidor. O Express identifica o tipo da requisição, como GET ou POST, e direciona para a rota correspondente previamente definida no código.

Após o processamento da requisição, o servidor retorna uma resposta ao cliente, que pode ser em formato HTML, JSON ou outro tipo de dado. O navegador então interpreta essa resposta e exibe o conteúdo ao usuário.

Um ponto importante é que, utilizando o HTTP/1.1, a conexão entre cliente e servidor pode permanecer aberta (conexão persistente), permitindo múltiplas requisições sem a necessidade de reabrir a conexão a cada interação. Isso torna a comunicação mais eficiente e rápida.

Além disso, cada requisição enviada pelo navegador inclui informações como o User-Agent, que permite ao servidor identificar características do cliente e, se necessário, adaptar a resposta conforme o dispositivo ou navegador utilizado.

4. Conexões Persistentes

Uma conexão é considerada persistente quando permanece aberta após a realização de uma requisição HTTP, permitindo que múltiplas requisições e respostas sejam realizadas utilizando a mesma conexão entre cliente e servidor. No protocolo HTTP/1.1, esse comportamento é adotado por padrão.

O uso de conexões persistentes traz diversas vantagens. Primeiramente, evita a necessidade de abrir e fechar uma nova conexão a cada requisição, reduzindo o custo associado ao estabelecimento de conexões TCP. Além disso, contribui para a redução da latência, uma vez que a comunicação ocorre de forma mais rápida com a conexão já estabelecida.

Outro benefício importante é a economia de recursos do servidor e da rede, já que há menor quantidade de conexões sendo criadas e encerradas. Como resultado, a experiência do usuário é aprimorada, especialmente em aplicações que realizam múltiplas requisições, como o carregamento de imagens, scripts e chamadas a APIs.

5. Conexões Não Persistentes

As conexões não persistentes representam o modelo de comunicação utilizado em versões anteriores do protocolo HTTP, como o HTTP/1.0 [Berners-Lee, T. and Fielding, R. and Nielsen, H. 1996], onde cada requisição exige o estabelecimento de uma nova conexão entre cliente e servidor.

Nesse modelo, o funcionamento ocorre da seguinte forma: o cliente inicia uma conexão com o servidor, envia uma requisição (como GET ou POST), o servidor processa essa requisição e retorna a resposta, e, em seguida, a conexão é imediatamente encerrada.

Esse processo, apesar de simples, apresenta desvantagens significativas, como o aumento da latência e o maior consumo de recursos, devido à necessidade de criar e finalizar conexões repetidamente para cada interação entre cliente e servidor.

6. Rotas e Tipos de Requisição

Em aplicações web baseadas em Node.js com o uso do framework Express, as rotas são responsáveis por definir como o servidor responde a diferentes requisições feitas pelo cliente. Cada rota está associada a um endereço (URL) e a um método HTTP específico, determinando a ação que será executada quando uma requisição é recebida.

Os principais métodos HTTP utilizados neste projeto são o GET e o POST. O método GET é utilizado para solicitar dados ao servidor, sem causar alterações no estado da aplicação. Ele é comumente empregado para recuperar informações, como páginas HTML ou dados em formato JSON. Já o método POST é utilizado para enviar dados ao servidor, geralmente com o objetivo de criar ou modificar recursos, sendo amplamente utilizado em formulários e envio de informações.

No Express, a definição de rotas é realizada de forma simples e intuitiva, por meio de funções que associam um caminho a um método HTTP. Por exemplo, uma rota definida como `app.get('/')` indica que o servidor responderá a requisições GET feitas para a raiz da aplicação. Da mesma forma, uma rota como `app.post('/dados')` permite o recebimento de dados enviados pelo cliente.

Além disso, as rotas podem ser organizadas de forma estruturada, seguindo padrões como APIs RESTful, onde cada endpoint representa um recurso da aplicação. Nesse modelo, diferentes métodos HTTP são utilizados sobre a mesma rota para representar operações distintas, como leitura, criação, atualização e remoção de dados.

Outro aspecto importante é o tratamento das requisições recebidas. O Express fornece objetos como `req` (requisição) e `res` (resposta), que permitem acessar informações enviadas pelo cliente, como parâmetros, corpo da requisição e cabeçalhos, além de possibilitar o envio de respostas apropriadas ao cliente.

Dessa forma, o uso adequado de rotas e métodos HTTP é fundamental para a organização e funcionamento da aplicação, permitindo uma comunicação clara, eficiente e estruturada entre cliente e servidor [Fielding 2000].

Tabela 1. Comportamento das requisições na aplicação

URL	Método HTTP	Comportamento
Qualquer rota	GET	Registra informações da requisição e retorna JSON
Qualquer rota	POST	Captura o corpo da requisição e retorna JSON
Qualquer rota	Outros	Processa e retorna resposta padrão em JSON

Diferentemente de aplicações estruturadas com múltiplas rotas específicas, o servidor desenvolvido neste projeto utiliza um único manipulador de requisições, sendo capaz de processar qualquer URL acessada pelo cliente. Independentemente do método HTTP utilizado, o servidor registra informações relevantes da requisição, como método, URL e cabeçalhos, além de capturar o corpo da mensagem quando presente.

A Tabela 1 resume o comportamento geral da aplicação para diferentes tipos de requisição.

7. Engenharia Reversa

A engenharia reversa é o processo de análise de um sistema com o objetivo de compreender sua estrutura, funcionamento e comportamento interno a partir de sua observação externa, geralmente sem acesso ao código-fonte original. Essa prática é amplamente utilizada em áreas como segurança da informação, interoperabilidade de sistemas e análise de protocolos de comunicação. Segundo [Eilam 2005], a engenharia reversa permite a extração de conhecimento a partir de artefatos existentes, sendo uma ferramenta essencial para a compreensão de sistemas complexos.

No contexto de aplicações web, a engenharia reversa é frequentemente aplicada na análise da comunicação entre cliente e servidor, que ocorre por meio do protocolo HTTP. De acordo com [Fielding 2000], a interação entre sistemas distribuídos na web é baseada em requisições e respostas, o que possibilita a observação detalhada do comportamento de uma aplicação por meio do tráfego de rede.

Neste projeto, a engenharia reversa pode ser realizada utilizando ferramentas como o painel de desenvolvedor dos navegadores (DevTools), que permitem monitorar e inspecionar todas as requisições HTTP realizadas pelo cliente. Por meio dessas ferramentas, é possível analisar elementos como métodos de requisição (GET, POST), URLs acessadas, parâmetros enviados, códigos de status retornados e cabeçalhos HTTP.

A análise dos cabeçalhos HTTP desempenha um papel fundamental nesse processo. Informações como o campo *User-Agent* permitem identificar características do cliente, enquanto outros cabeçalhos, como *Content-Type* e *Authorization*, revelam detalhes sobre o formato dos dados e possíveis mecanismos de autenticação utilizados pela aplicação. Segundo [Fielding, R. and Reschke, J. 2014], os cabeçalhos HTTP são essenciais para a correta interpretação das mensagens trocadas entre cliente e servidor.

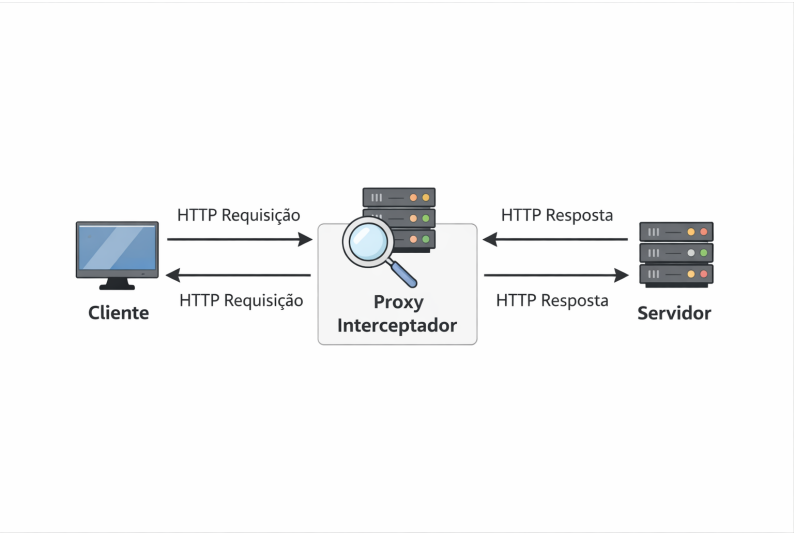

Figura 1. Fluxo de interceptação de requisições HTTP utilizando um proxy intermediário, permitindo análise e modificação das mensagens entre cliente e servidor.

Além disso, a engenharia reversa possibilita a identificação das rotas disponíveis em uma aplicação web. Ao observar as requisições realizadas, é possível mapear endpoints, inferindo os tipos de operações suportadas pelo servidor. Esse processo, conhecido como *API mapping*, é amplamente utilizado para documentar sistemas que não possuem especificação formal ou documentação disponível.

Outro aspecto importante é a análise do formato das respostas retornadas pelo servidor, geralmente estruturadas em JSON. A partir dessas respostas, pode-se compreender a organização dos dados, suas relações e possíveis regras de negócio implementadas no backend. Essa abordagem permite reconstruir parcialmente a lógica da aplicação, contribuindo para testes, integração com outros sistemas e auditorias.

Entretanto, é importante destacar que a engenharia reversa deve ser conduzida de maneira ética e legal. Conforme discutido em [Chikofsky and Cross 1990], o uso dessa técnica deve respeitar restrições legais, direitos autorais e políticas de uso dos sistemas analisados, sendo recomendada principalmente em contextos acadêmicos, educacionais ou com autorização explícita.

Portanto, a aplicação da engenharia reversa neste projeto permite uma compreensão mais aprofundada da comunicação baseada em HTTP, das rotas implementadas no servidor Node.js e do comportamento geral da aplicação. Essa análise contribui significativamente para o aprendizado prático dos conceitos de redes de computadores e desenvolvimento web, além de reforçar a importância da observação e interpretação de sistemas reais.

8. Trabalhos Futuros

Como continuidade deste projeto, diversas melhorias podem ser implementadas com o objetivo de tornar a aplicação mais robusta, escalável e alinhada às práticas modernas de desenvolvimento web.

Uma das principais evoluções consiste na implementação de um sistema de autenticação e autorização, utilizando mecanismos como tokens JWT (*JSON Web Tokens*). Essa abordagem permite controlar o acesso a recursos da aplicação de forma segura, garantindo que apenas usuários autenticados possam realizar determinadas operações.

Outra melhoria relevante é a integração com um banco de dados, como o MongoDB ou o MySQL, possibilitando a persistência de dados. Isso permitiria transformar a aplicação em um sistema mais completo, com armazenamento de informações e suporte a operações CRUD (*Create, Read, Update, Delete*).

Além disso, a evolução para uma arquitetura baseada em APIs RESTful pode tornar a aplicação mais modular e interoperável. Segundo [Fielding 2000], esse modelo arquitetural promove escalabilidade e separação de responsabilidades entre cliente e servidor, sendo amplamente adotado em sistemas distribuídos modernos.

Também é possível considerar a utilização de containers, como o Docker, para facilitar o processo de deploy e garantir consistência entre diferentes ambientes de execução. Essa abordagem contribui para a portabilidade e simplifica a distribuição da aplicação.

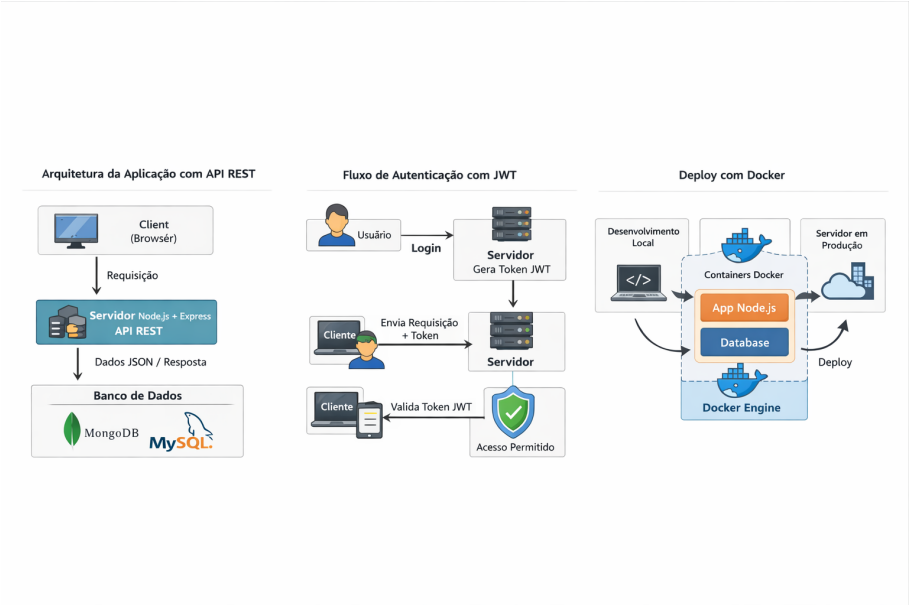

Figura 2. Ilustração de possíveis futuras aplicações.

Por fim, melhorias relacionadas ao desempenho podem ser exploradas, como o uso de balanceamento de carga e otimizações no tratamento de requisições HTTP. A adoção de práticas como cache de respostas e compressão de dados pode reduzir significativamente o tempo de resposta do servidor, melhorando a experiência do usuário.

Referências

Berners-Lee, T. and Fielding, R. and Nielsen, H. (1996). Hypertext transfer protocol – http/1.0. RFC 1945.

Chikofsky, E. J. and Cross, J. H. (1990). Reverse engineering and design recovery: A taxonomy. *IEEE Software*, 7(1):13–17.

Eilam, E. (2005). *Reverse Engineering*. Wiley Publishing.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures.

Fielding, R. and Reschke, J. (2014). Hypertext transfer protocol (http/1.1): Message syntax and routing. RFC 7230.

Tanenbaum, A. S. and Wetherall, D. J. (2011). *Computer Networks*. Pearson, 5 edition.